

AS 2: Scrape one feature in one page

Scrapy

- One of only two python-based frameworks for scrape (the other is beautiful soup)
- A full-fledged web scraping framework
 - Built-in request(), can handle **multiple pages**
 - Asynchronous networking library, **faster, production-level scale**
 - Export the data in a number of formats like CSV
 - **Extensions** for tasks like cookie handling, user-agent spoofing, restricting crawl depth, and others, as well as an API for easily building your own additions
- Command-line based
- Let's build a scrapy spider to crawl some data from tumblr

Scrape Tumblr data

- **Part 1:** Let's start with scraping **one feature** from **a single page**
- Challenging level: ★
- Install scrapy package
 - Open your **terminal**

```
pip install scrapy
```

In terminal (command line)

- Step 1: Setup a scrapy project

```
scrapy startproject tumblr
```

#project name, a.k.a., folder name

- Step 2: Change directory to the project

```
cd tumblr
```

- Step 3: Create a blank spider

```
scrapy genspider quote https://onequoteperday.tumblr.com/
```

#spider name

#where it first visits or you want to put this spider to crawl

In Python editor

- Step 4: open the blank spider
 - It is located at \user\tumblr\spiders\quote.py

```
import scrapy

class QuoteSpider(scrapy.Spider):
    name = "quote"           #spider name
    allowed_domains = ["onequoteperday.tumblr.com"]
    start_urls = ["https://onequoteperday.tumblr.com/"]

    def parse(self, response):
        pass #do nothing
```

the domains that
the spider is
allowed scrape

where this spider
start to scrape

In Python editor

- Step 5: Edit the parse function to build and empower your spider

```
def parse(self, response):  
    quotes = response.xpath('//div[@class="post quote" or "post text"]/span/text()).extract()
```

#response.xpath: a **selector**, used to navigate and select elements from HTML or XML documents.

Simpler alternative:
response.css

#specify the **path** in xpath format
(1) //: start from the <html> tag
(2) div[@class="post quote" or "post text"]: until <div> tag whose class is class="post quote" or class="post text"
(3) /span: find the tag under the <div> tag above

#**identify**
the text
after the

tag

#**extract**
every
instance
following
the xpath
into a []

```
print(quotes)
```

```
for i in quotes:  
    yield {'Quote': i}
```

#transform [] into {}

Save it after all!!!

Back to terminal

- Step 6: Run the code

- In your **terminal**

- save the {} as CSV

- XML

- JSON

```
scrapy crawl quote
```

```
scrapy crawl quote -o demo_output.csv
```

```
scrapy crawl quote -o demo_output.xml
```

```
scrapy crawl quote -o demo_output.json
```

File name

- Check your output at your project folder

Mini Group Project (MGP) AS1

- Visit <https://quotes.toscrape.com/>
- Create a new project for our mini- project series
- Create a new spider in this project to scrape all the quotes on the first page
- Save it in a CSV file
- Create another spider to scrape all the 'tags' on the first page
- Save it in a CSV file

